

Tervezési Minták

A tervezési minták olyan általánosan bevált megoldások visszatérő szoftverfejlesztési problémákra, amelyek segítenek strukturáltabban, rugalmasabban és karbantarthatóbban írni a kódot. Nem konkrét kódrészletek, hanem sablonok, elvek, amit különböző helyzetekben alkalmazhatunk.

1. Építő (Builder)

Az Építő minta lehetővé teszi komplex objektumok lépésről lépésre történő létrehozását. Ugyanazzal a konstrukciós kóddal az objektum különböző típusait és reprezentációit állíthatjuk elő.

Probléma: Képzeljük el, hogy egy komplex objektumot kell létrehozni (pl. egy Ház osztályt). Kezdetben csak falak és tető kellenek. De mi van, ha később kell garázs, úszómedence, szobrok vagy kert? A hagyományos megoldásban vagy létrehozunk rengeteg alosztályt (HazGarazzsal, HazMedencevel), vagy ami még rosszabb Java-ban egy óriási konstruktort írunk rengeteg paraméterrel, amelyek többsége null lesz az esetek nagy részében (ezt hívják "teleszkópikus konstruktor" problémának).

Megoldás: A minta kiszervezi az objektum konstrukcióját a saját osztályunkból egy különálló "builder" objektumba. Az objektumot lépésenként rakjuk össze pl. `buildWalls()`, `buildRoof()`, és csak azokat a lépéseket hívjuk meg, amelyekre valóban szükség van.

2. Stratégia (Strategy)

A Stratégia minta lehetővé teszi, hogy algoritmusok egy családját definiáljuk, külön osztályokba helyezzük őket, és ezeket az objektumokat felcserélhetővé tegyük.

Probléma: Készítünk egy navigációs alkalmazást. Eleinte csak autós útvonalat tervez. Később hozzáadjuk a gyalogos, majd a tömegközlekedési, végül a kerékpáros útvonalat. A főkédben egy hatalmas, bonyolult if-else vagy switch szerkezet jön létre, amely eldönti, melyik algoritmust futtassa. Ha változtatni kell az egyik algoritmuson, az egész osztályt módosítani kell, ami növeli a hibák kockázatát.

Megoldás: A minta azt javasolja, hogy minden egyes útvonaltervezési módszert szervezz ki egy-egy saját osztályba (ezek a "stratégiák"). A fő navigációs osztály nem tudja, hogyan működik a keresés, csak van egy hivatkozása egy `UtvonalStrategia` interfészre, és a munka elvégzését átadja (delegálja) az éppen aktuális stratégia-objektumnak.

3. Homlokzat (Facade)

A Homlokzat mint a egy egyszerűsített interfészt biztosít egy összetett könyvtárhoz, keretrendszerhez vagy osztálykészlethez.

Probléma: A kódnak egy bonyolult, külső könyvtárral kell dolgoznia, amely több tucat osztályból áll. Hogy elvégezzünk egy feladatot, inicializálni kell ezeket, beállítani a függőségeiket és helyes sorrendben meghívni a metódusaikat. Ennek eredményeként a kód üzleti logikája szorosan összefonódik a külső könyvtár részleteivel, ami nehezen karbantarthatóvá teszi a rendszert.

Megoldás: Létrehozunk egy "Homlokzat" osztályt, amely egy egyszerű felületet kínál a kliensnek (pl. egyetlen `convertVideo()` metódust). Ez a homlokzat a háttérben elvégzi a "piszkos munkát": példányosítja a megfelelő osztályokat és kezeli a bonyolult hívásokat. A program többi része csak a homlokzatot ismeri.